



تمرین سوم

بخش اول: سوالات تئوری و تحلیلی

سوال ۱: تحلیل ریاضیاتی محوشدگی گرادیان و نقش Cell State در LSTM

با توجه به معادلات شبکه RNN استاندارد که در آن $s_t = \tanh(Ux_t + Ws_{t-1})$ است:

الف) نشان دهید که در الگوریتم BPTT، عبارت $\frac{\partial s_t}{\partial s_{t-d}}$ شامل ضرب زنجیره‌ای ماتریس‌های ژاکوبین است و ثابت کنید که اگر مقادیر ویژه ماتریس وزن W به اندازه کافی کوچک باشند، این گرادیان با افزایش d به صورت نمایی به سمت صفر میل می‌کند.

ب) حال شبکه LSTM را در نظر بگیرید. با استفاده از معادله به‌روزرسانی حافظه $\tilde{C}_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$ ، مشتق $\frac{\partial C_t}{\partial C_{t-1}}$ را محاسبه کنید. از نظر ریاضیاتی توضیح دهید چگونه وجود گیت فراموشی (f_t) و عملگر جمع، از ضرب متوالی مقادیر کوچکتر از یک جلوگیری کرده و مشکل محوشدگی گرادیان را حل می‌کند.

سوال ۲: استخراج معادلات پس‌انتشار (Backward Pass) برای گیت Reset در GRU

معادلات یک واحد GRU به صورت زیر است:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t])$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

فرض کنید گرادیان خطا نسبت به خروجی پنهان در زمان t ، یعنی $\delta h_t = \frac{\partial E}{\partial h_t}$ در دسترس است. رابطه دقیق $\delta W_r = \frac{\partial E}{\partial W_r}$ (گرادیان ماتریس وزن‌های گیت Reset) را بر حسب δh_t و h_{t-1} و سایر متغیرهای میانی محاسبه کنید. تمامی مراحل مشتق‌گیری زنجیره‌ای (Chain Rule) را به صورت کامل و با در نظر گرفتن عملگر ضرب درایه به درایه (Hadamard Product) بنویسید.

بخش دوم: پیاده‌سازی و کدنویسی

سوال ۳: پیاده‌سازی Coupled Input-Forget Gate LSTM از پایه با NumPy

در جزوه به نسخه‌ای از LSTM اشاره شده است که در آن گیت‌های ورودی و فراموشی به هم متصل هستند ($f_t = 1 - i_t$) و معادله به‌روزرسانی حافظه به شکل $C_t = (1 - i_t) \odot C_{t-1} + i_t \odot \tilde{C}_t$ در می‌آید. این تغییر، تعداد پارامترهای شبکه را کاهش می‌دهد.

الف) یک کلاس در پایتون با استفاده از کتابخانه NumPy بنویسید که مرحله Forward Pass این نوع LSTM را به صورت کامل پیاده‌سازی کند. استفاده از هرگونه فریم‌ورک یادگیری عمیق مانند PyTorch یا TensorFlow ممنوع است.

ب) کلاسی که نوشتید را روی یک توالی زمانی مصنوعی (Synthetic Time Series) شامل ۵۰ گام زمانی اعمال کنید. وزن‌ها را به صورت تصادفی (Random Initialization) مقداردهی کنید و خروجی h_t را به ازای هر

گام زمانی استخراج کنید. پیاده‌سازی شما باید از نظر محاسبات ماتریسی بهینه باشد و ابعاد ماتریس‌ها (Matrix Dimensions) کاملاً تطابق داشته باشند.

سوال ۴: پیاده‌سازی برداری پردازش دسته‌ای (Batched Processing) و پس‌انتشار در زمان (BPTT) با ترفندهای بهینه‌سازی

همان‌طور که در اسلاید ۲۷ فایل جزوه اشاره شده است، در پیاده‌سازی حرفه‌ای و بهینه شبکه‌های بازگشتی (نظیر RNN و LSTM)، به جای محاسبه مجزای عباراتی مانند Wx_t و Uh_{t-1} ، بردارها را به هم متصل (Concatenate) کرده و از یک ضرب ماتریسی واحد به فرم $W_{combined} \cdot [x_t, h_{t-1}]^T$ استفاده می‌شود. این ترفند سرعت اجرا را به شدت افزایش می‌دهد. یک اسکریپت پایتون (قابل اجرا در محیط Jupyter Notebook یا Google Colab) با استفاده از NumPy خالص بنویسید که موارد زیر را شامل شود:

الف) یک کلاس Vanilla RNN از پایه طراحی کنید که برخلاف حالت ساده، قابلیت پردازش دسته‌ای (Mini-batch Processing) را داشته باشد. تنسور ورودی باید دارای ابعاد (Batch Size, Sequence Length, Features) باشد. در متد forward حتماً از ترفند Matrix Concatenation برای ادغام متغیرهای ورودی و حالت پنهان (Hidden State) استفاده کنید.

ب) متد backward را برای اجرای کامل BPTT (Backpropagation Through Time) پیاده‌سازی کنید. با توجه به محدودیت این شبکه‌ها که در جزوه ذکر شده، برای جلوگیری از مشکل انفجار گرادیان (Exploding Gradients)، تکنیک Gradient Clipping را به صورت دستی درون کد خود پیاده کرده و قبل از به‌روزرسانی وزن‌ها (Weight Update) اعمال کنید.

ج) یک مجموعه داده مصنوعی بسیار سبک با ساختار Many-to-One ایجاد کنید (مثلاً طبقه‌بندی یک دنباله باینری بر اساس تعداد زوج یا فرد بودن عدد یک). مدل خود را روی این داده‌ها با استفاده از حلقه آموزش (Training Loop) و گرادیان کاهشی تصادفی (SGD) آموزش دهید.

د) پیچیدگی کد باید به گونه‌ای بهینه شده باشد که آموزش ۵۰۰ Epoch از این شبکه روی یک سخت‌افزار معمولی (صرفاً با CPU) تنها در چند ثانیه اجرا شود و در نهایت نمودار کاهش خطای آنتروپی متقاطع (Cross-Entropy Loss) با کتابخانه Matplotlib رسم گردد.

سوال ۵: پیاده‌سازی حرفه‌ای و ترفندهای آموزش RNN‌ها در PyTorch

در کاربردهای واقعی، توالی‌های ورودی (Sequential Data) غالباً طول‌های متفاوتی دارند و آموزش شبکه‌های عمیق بازگشتی با چالش‌هایی نظیر انفجار گرادیان (Exploding Gradients) و محاسبات هدررفته روی پدینگ‌ها (Padding) مواجه است. با توجه به مفاهیم شبکه‌های دوطرفه (BiRNN) و منظم‌سازی (Regularization) در فایل جزوه، یک برنامه پایتون مبتنی بر فریم‌ورک PyTorch بنویسید که شامل دو بخش زیر باشد:

الف) معماری مدل (Handling Variable-Length Sequences): یک کلاس به نام RobustBiLSTM از نوع nn.Module پیاده‌سازی کنید. این مدل باید شامل یک لایه Bidirectional LSTM چندلایه (Stacked) همراه با مکانیزم Dropout باشد. ترفند الزامی: برای جلوگیری از انجام محاسبات روی مقادیر Padding و افزایش سرعت روی سخت‌افزارهای معمولی، موظف هستید در متد forward از توابع pack_padded_sequence و pad_packed_sequence استفاده کنید. خروجی تابع باید بردار ویژگی نهایی (حاصل اتصال یا Concatenation آخرین حالت پنهان مسیر رفت و برگشت) برای یک تسک دسته‌بندی توالی (Sequence Classification) باشد.

ب) حلقه آموزش پایدار (Stable Training Loop & Tricks): کدی برای حلقه آموزش (Training Loop) روی یک دیتاست مصنوعی (Synthetic Dataset) بسیار کوچک بنویسید (تا روی CPU لپ‌تاپ به راحتی اجرا شود). در این حلقه، پیاده‌سازی سه تکنیک حرفه‌ای زیر الزامی است:

۱. برش گرادیان (Gradient Clipping): برای جلوگیری از مشکل انفجار گرادیان، از torch.nn.utils.clip_grad_norm_ استفاده کنید.

۲. مقداردهی اولیه متعامد (Orthogonal Initialization): تابعی بنویسید که پیش از شروع آموزش، ماتریس وزن‌های تکرارشونده‌ی (Recurrent Weights) لایه LSTM را به صورت متعامد (Orthogonal) مقداردهی کند تا به جریان بهتر گرادیان در طول زمان (BPTT) کمک کند.
۳. مدیریت State: نشان دهید چگونه حالت پنهان (Hidden State) را در هر بچ (Batch) به درستی Detach یا ریست می‌کنید تا گراف محاسباتی بیش از حد رشد نکند و حافظه سیستم (RAM/VRAM) پر نشود.